```
sbt "chapter16/runMain example.locator.grpc.LocatorServer 10"
```

Once the gRPC service continues serving `Locations`, the result in `RestartSource` is as expected. It picks the initial element of the service, and in the following seconds, the test continues printing the locations consumed:

```
[11:09:23,679] [WARN] [akka.stream.scaladsl.RestartWithBackoffSource] []
➥[SampleSpec-akka.actor.default-dispatcher-3] - Restarting stream due
➥to failure ...
...                                                          ❶
Location(10.0,-3.701101,UnknownFieldSet(Map()))              ❷
Location(11.0,-3.701101,UnknownFieldSet(Map()))
Location(12.0,-3.701101,UnknownFieldSet(Map()))
...                                                          ❸
```

❶ More failing retries

❷ Consumes one element per second

❸ More consecutive Location elements

Let's go into a little more detail about how the `RestartSource` is used in combination with the gRPC service:

```
class LocatorServiceSpec ... {
  ...

      val restartSource = RestartSource.withBackoff(settings) { () =>
        Source.futureSource {
          val responseStream: Source[Location, NotUsed] =
            client.follow()
          Future(responseStream)
      }}
  ...
}
```

Once you pass the settings to the `RestartSource`, you need a source that deals with futures—hence the name `Source.futureSource`. After all, you are calling a service that